

2.1 [5] <§2.2> For the following C statement, what is the corresponding MIPS assembly code? Assume that the variables `f`, `g`, `h`, and `i` are given and could be considered 32-bit integers as declared in a C program. Use a minimal number of MIPS assembly instructions.

`f = g + (h - 5);`

2.2 [5] <§2.2> For the following MIPS assembly instructions above, what is a corresponding C statement?

`add f, g, h` $f \leftarrow g+h$
`add f, i, f` $f \leftarrow i+f$ $> f = g+h+i$

2.1 `addi f, h, -5` (note, no `subi`)
`add f, f, g` MIPS 沒有 `subi`

2.2 $f = g + h + i$

2.3 [5] <§§2.2, 2.3> For the following C statement, what is the corresponding MIPS assembly code? Assume that the variables **f, g, h, i, and j** are assigned to registers **\$s0, \$s1, \$s2, \$s3, and \$s4**, respectively. Assume that the base address of the arrays **A** and **B** are in registers **\$s6 and \$s7**, respectively.

$B[8] = A[i-j];$
 S7 S6 $S6 + (i-j) \times 4$
 → 每個陣列idx+1時 先算 $(i-j) \times 4$
 mem 位置+4! 後+上 S6

2.4 [5] <§§2.2, 2.3> For the MIPS assembly instructions below, what is the corresponding C statement? Assume that the variables **f, g, h, i, and j** are assigned to registers **\$s0, \$s1, \$s2, \$s3, and \$s4**, respectively. Assume that the base address of the arrays **A** and **B** are in registers **\$s6 and \$s7**, respectively.

sll	\$t0, \$s0, 2	# \$t0 = f * 4	f	g	h	i	j	A	B
add	\$t0, \$s6, \$t0	# \$t0 = &A[f]							
sll	\$t1, \$s1, 2	# \$t1 = g * 4	s0	s1	s2	s3	s4	s6	s7
add	\$t1, \$s7, \$t1	# \$t1 = &B[g]							
lw	\$s0, 0(\$t0)	# f = A[f]							
addi	\$t2, \$t0, 4	t2 = &A[f+1]							
lw	\$t0, 0(\$t2)	t0 = A[f+1]							
add	\$t0, \$t0, \$s0	t0 = A[f+1] + A[f]							
sw	\$t0, 0(\$t1)	B[g] = A[f+1] + A[f]							

$$B[8] = A[i-j]$$

2.3

$\begin{matrix} i & j \\ \text{sub } \$t0, \$s3, \$s4 & t0 \leftarrow i-j \end{matrix}$
 $\begin{matrix} f & g & h & i & j \\ s0 & s1 & s2 & s3 & s4 \end{matrix}$

$\text{sll } \$t0, \$t0, 2$ $t0$ 左移2bit $t0 = (i-j) \times 4$

$\text{add } \$t0, \$t0, \$s6$ $t0 = t0 + s6 \rightarrow$ 算出A陣列的地址了

$\text{lw } \$t1, 0(\$t0)$ $t1 = M[t0]$ ($t1 = A[i-j]$)

$\text{sw } \$t1, 32(\$s7)$ $B[8] = A[i-j]$ ($B[8] = t1$)
 8×4

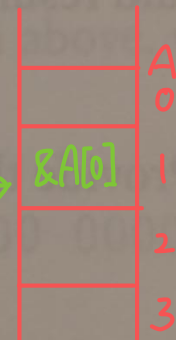
2.4 $B[g] = A[f] + A[1+f];$

2.9 [5] <§§2.2, 2.3> Translate the following C code to MIPS. Assume that the variables `f`, `g`, `h`, `i`, and `j` are assigned to registers `$s0`, `$s1`, `$s2`, `$s3`, and `$s4`, respectively. Assume that the base address of the arrays `A` and `B` are in registers `$s6` and `$s7`, respectively. Assume that the elements of the arrays `A` and `B` are 4-byte words:

`B[8] = A[i] + A[j];`
 s_3 s_4

2.10 [5] <§§2.2, 2.3> Translate the following MIPS code to C. Assume that the variables `f`, `g`, `h`, `i`, and `j` are assigned to registers `$s0`, `$s1`, `$s2`, `$s3`, and `$s4`, respectively. Assume that the base address of the arrays `A` and `B` are in registers `$s6` and `$s7`, respectively.

```
addi $t0, $s6, 4   $t_0 = \&A[1]$ 
add  $t1, $s6, $0   $t_1 = \&A[0]$ 
sw   $t1, 0($t0)
lw   $t0, 0($t0)   $t_0 = \&A[0]$ 
add  $s0, $t1, $t0   $s_0 = \&A[0] + \&A[0]$ 
```



$f = 2 \&A$ ($f = 2 \&A[0]$)

2.9

```

sll $t0, $s3, 2    # t0 = i*4
add $t0, $t0, $s6  # t0 = addr A[i] (&A[i])
lw $t1, 0($t0)     # t1 = A[i] Get A[i]
sll $t0, $s4, 2    # t0 = j*4
add $t0, $t0, $s6  # t0 = addr A[j]
lw $t2, 0($t0)     # t2 = A[j] Get A[j]
add $t2, $t1, $t2  # t2 = A[i] + A[j]
sw $t2, 32($s7)    # B[8] = A[i] + A[j]
                  8*4      B[8] = t2 (M[S7+32]=t2)

```

f g h i j A B
s0 s1 s2 s3 s4 s6 s7

2.10 f = 2*(&A);



```

EX: sll $t0, $s1, 2 # $t0 <- 4*g
    add $t0, $t0, $s7 # $t0 <- Addr(B[g])
    lw $t0, 0($t0) # $t0 <- B[g]
    addi $t0, $t0, 1 # $t0 <- B[g]+1
    sll $t0, $t0, 2 # $t0 <- 4*(B[g]+1) → 变成 Array 的 idx
    add $t0, $t0, $s6 # t0 = addr A[B[g]+1]
    lw $s0, 0($t0) # f <- A[B[g]+1]

```


2.22 [5] <\$2.6> For the following C statement, write a minimal sequence of MIPS assembly instructions that does the identical operation. Assume $\$t1 = A$, $\$t2 = B$, and $\$s1$ is the base address of C.

$A = C[0] \ll 4;$

2.23 [5] <\$2.7> Assume $\$t0$ holds the value $0x00101000$. What is the value of $\$t2$ after the following instructions?

```
    slt    $t2, $0, $t0 (t2=1)
    bne    $t2, $0, ELSE
    j      DONE
ELSE: addi  $t2, $t2, 2 (t2=t2+2)
DONE:
```

Handwritten notes:
A blue arrow points from the value 0 in the first instruction to the register \$t0.
A red arrow points from the label ELSE to the instruction addi \$t2, \$t2, 2.
Below the ELSE label, it is written: $t2=3$.

Handwritten notes:
slt \$t2, \$0, \$t0
if (\$t2 < 0) \$t2 = 1
else \$t2 = 0

2.22 lw \$t3, 0(\$s1)
sll \$t1, \$t3, 4

C
c[0]

2.23 \$t2 = 3

2.26 Consider the following MIPS loop:

```
      | ← 0      10 9
LOOP: slt  $t2, $0, $t1
      beq  $t2, $0, DONE 不跳
      subi $t1, $t1, 1  (t1=9) 8
      addi $s2, $s2, 2  (s2=2) 4
      j    LOOP          10 次
```

DONE:

若 $t1=10$

做了 50 次指令

最後一次時, 先做比較 (slt)

後跳走 (beq) → 總共 50 + 2 次

做 10 次

2.26.1 [5] <§2.7> Assume that the register $\$t1$ is initialized to the value 10. What is the value in register $\$s2$ assuming $\$s2$ is initially zero?

$s2$ 變成 20

2.26.2 [5] <§2.7> For each of the loops above, write the equivalent C code routine. Assume that the registers $\$s1$, $\$s2$, $\$t1$, and $\$t2$ are integers A, B, i, and temp, respectively.

2.26.3 [5] <§2.7> For the loops written in MIPS assembly above, assume that the register $\$t1$ is initialized to the value N. How many MIPS instructions are executed?

$5N+2$

2.26

2.26.1 20

2.26.2 `i = 10;`
 `do {`
 `B += 2;`
 `i = i - 1;`
 `} while ( i > 0)`

2.26.3 $5 * N + 2$

2.27 [5] <\$2.7> Translate the following C code to MIPS assembly code. Use a minimum number of instructions. Assume that the values of *a*, *b*, *i*, and *j* are in registers *\$s0*, *\$s1*, *\$t0*, and *\$t1*, respectively. Also, assume that register *\$s2* holds the base address of the array *D*.

```
for(i=0; i<a; i++)  
    for(j=0; j<b; j++)  
        D[4*j] = i + j;
```

2.27

```
addi $t0, $0, 0  $i=0$ 
```

```
beq $0, $0, TEST1 無條件跳到 TEST1
```

```
LOOP1: addi $t1, $0, 0  $j=0$ 
```

```
beq $0, $0, TEST2 → 無條件跳到 TEST2
```

```
LOOP2: add $t3, $t0, $t1 →  $t3 = i + j$ 
```

```
sll $t2, $t1, 4 →  $j \times 4$  得 idx, idx 還要  $\times 4$  才得 mem 位置。  
 $t2 = j \times 16$ 
```

```
add $t2, $t2, $s2 → 做  $D[4*j] = i + j$ 
```

```
sw $t3, 0($t2) →  $t2 = \&D[4*j]$  (算出位址)
```

```
addi $t1, $t1, 1 →  $t3$  內容送  $t2$  mem 位置 ( $D[4*i] = i + j$ )
```

```
TEST2: slt $t2, $t1, $s1  $b$  →  $j++$ 
```

```
bne $t2, $0, LOOP2 | 和  $b$  比,  $\neq$  跳 LOOP2 | 若  $j$  不小於  $b$   
 $t2 = 0$   
 $0, 0$  相等, 不跳  
繼續執行
```

```
addi $t0, $t0, 1 →  $i++$ 
```

```
TEST1: slt $t2, $t0, $s0  $a$ 
```

```
bne $t2, $0, LOOP1
```

判斷外圈
是否 $< a$

和 0 比較
 $\neq$ 跳 LOOP1

a	b	i	j	D
$s0$	$s1$	$t0$	$t1$	$s2$

for ($i=0; i < a; i++$)

for ($j=0; j < b; j++$)

$D[4*j] = i + j$

若 i 不小於 b , $t2 = 0$

$0, 0$ 相等, 不跳, 繼續執行 (Loop 結束)